

HIGH-LEVEL DESIGN

Distributed SMS Service

A polyglot, event-driven messaging platform with a transactional outbox

Author: Krishna Thakur

Date: 11 June 2026

Version: 1.0

Java 21 · Spring Boot 3.2 · Go · Kafka · Redis · MongoDB · H2 · Docker

Table of Contents

1. Introduction & Overview
2. Goals & Non-Goals
3. System Architecture
4. Component Design
5. End-to-End Data Flow
6. The Outbox Pattern
7. Reliability & Idempotency
8. Data Model
9. Security
10. Technology Choices & Rationale
11. API Reference
12. Configuration Reference
13. Deployment
14. Known Limitations & Roadmap
15. Appendix & Glossary

1. Introduction & Overview

The Distributed SMS Service is a polyglot, event-driven platform that accepts SMS-send requests, delivers them to a vendor, and durably records every outcome. It is composed of two independently deployable microservices that never call each other directly — they communicate asynchronously through a Kafka topic.

The central design principle is **decoupling acceptance from work**: a request is durably accepted and acknowledged in milliseconds, while the actual vendor call and event publication happen in the background. This makes the system resilient to downstream outages and guarantees that each recipient is messaged exactly once.

1.1 At a glance

Service	Language / Stack	Role	Port
SMS Sender	Java 21, Spring Boot 3.2	Synchronous API; accepts requests, runs the outbox worker, publishes events	8080
SMS Store	Go (net/http, segmentio/kafka-go)	Async worker; consumes events, persists history, serves queries	8081
Demo Gateway	Go (stdlib reverse proxy)	Serves the UI, routes API calls, injects the admin key	3000

2. Goals & Non-Goals

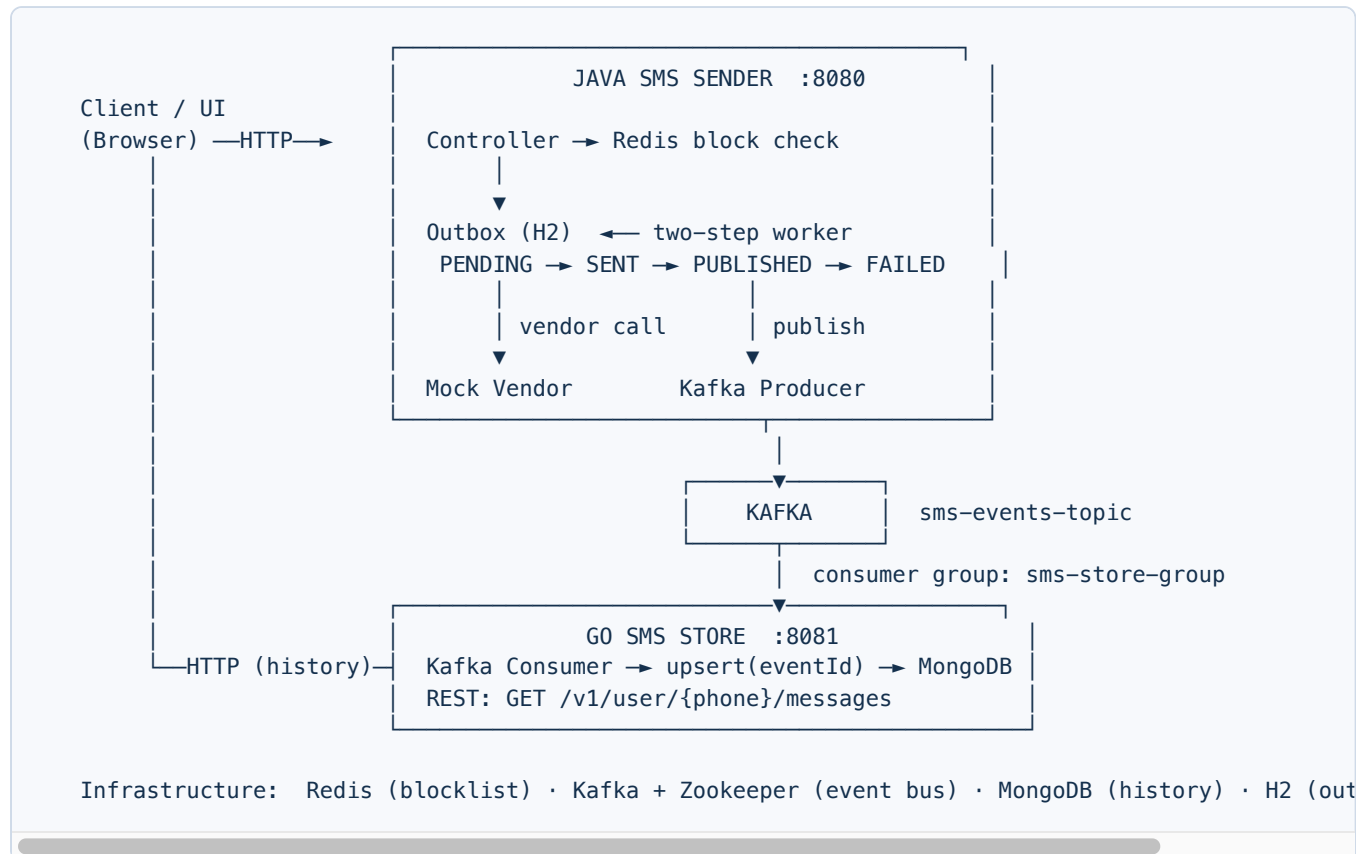
2.1 Goals

- **Durable acceptance** — once a request returns `202 Accepted`, it will never be lost.
- **Exactly-once to the recipient** — a retry or crash must never send the same SMS twice.
- **Exactly-once storage** — the history store must hold one record per request regardless of redelivery.
- **Resilience to outages** — a Kafka or store outage must not bring down the request path.
- **Operability** — stuck messages are quarantined and replayable, not lost or infinitely retried.
- **Protected admin surface** — privileged operations require authentication.

2.2 Non-Goals (for this iteration)

- Real vendor integration (a deterministic mock vendor stands in).
- Multi-tenancy / per-seller isolation (single-operator model for now).
- High-availability infrastructure (single-broker Kafka, single-node stores).
- User-facing authentication (admin uses a shared API key; see §14).

3. System Architecture



3.1 Why two services

The sender is **synchronous and user-facing** — it must respond fast and stay available. The store is **asynchronous and write-heavy** — it processes at its own pace. Splitting them lets each scale and fail independently. Kafka is the durable buffer between them, so the sender never blocks on, or fails because of, the store.

4. Component Design

4.1 Java SMS Sender

Layer	Responsibility
<code>SmsController</code>	REST endpoints: send, health, block/unblock/check
<code>OutboxController</code>	Admin: list outbox, list failed, replay
<code>SmsServiceImpl</code>	Redis block check; writes the outbox row; returns 202
<code>OutboxProcessor</code>	Scheduled worker; drives PENDING→SENT→PUBLISHED with backoff + dead-letter
<code>MockSmsVendorService</code>	Deterministic vendor; idempotent by key
<code>KafkaSmsEventPublisher</code>	Serializes the event and publishes synchronously (worker thread)
<code>ApiKeyAuthFilter</code>	Guards the admin surface with an API key

4.2 Go SMS Store

Package	Responsibility
<code>kafka</code>	Consumer loop; reads the topic, unmarshals, hands to the service
<code>services</code>	MongoDB persistence; idempotent upsert keyed by <code>eventId</code>
<code>handlers</code>	HTTP handlers for health and message history
<code>models</code>	Shared <code>SmsEvent</code> struct (json + bson tags)
<code>config</code>	Env-driven configuration with sensible defaults

4.3 Demo Gateway (Reverse Proxy)

A small Go reverse proxy serves the single-page UI and routes `/api/sender/*` to the Java service and `/api/store/*` to the Go service. Because the browser only ever sees one origin (`:3000`), CORS is avoided entirely. The proxy also **injects the admin API key** on sender-bound traffic, acting as a trusted gateway so the secret never lives in client code.

5. End-to-End Data Flow

5.1 Happy path (send → stored)

1. Client → POST /v1/sms/send {phoneNumber, message}
2. Sender: SISMEMBER blocked_users → not blocked
3. Sender: INSERT outbox row (status=PENDING, id=UUID)
4. Sender: return 202 Accepted {status:"ACCEPTED", messageId:UUID} ← fast, durable
5. Worker (≤1s later): PENDING row → vendor.send(id, phone, msg) → SUCCESS
6. Worker: UPDATE row status=SENT, vendorMessageId, attempts=0
7. Worker (same tick): SENT row → publish to Kafka (blocking, 5s ack)
8. Worker: UPDATE row status=PUBLISHED
9. Go: ReadMessage → unmarshal SmsEvent
10. Go: ReplaceOne(filter={eventId}, upsert=true) → MongoDB
11. UI History: GET /v1/user/{phone}/messages → event visible

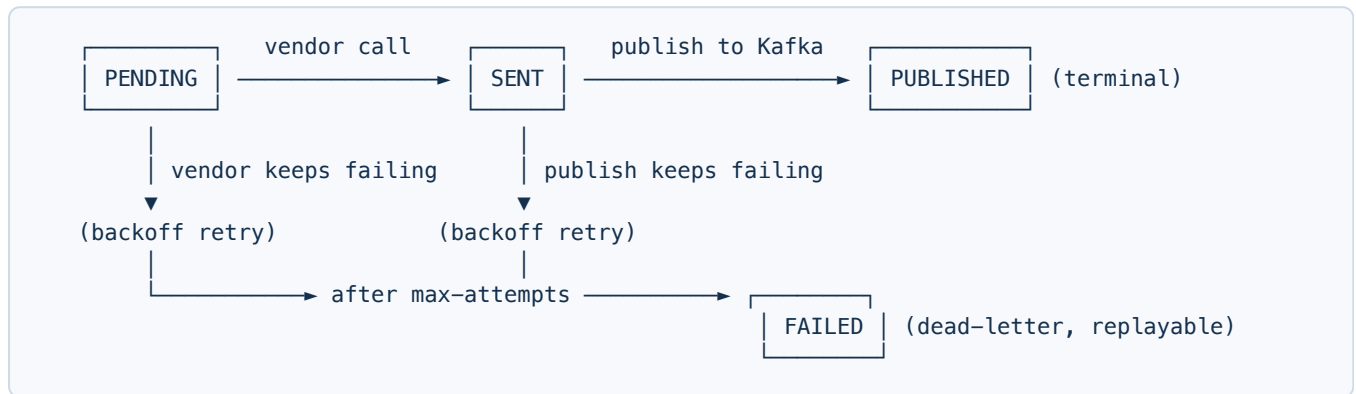
5.2 Blocked path

1. Client → POST /v1/sms/send
2. Sender: SISMEMBER blocked_users → BLOCKED
3. Sender: throw BlockedUserException → 403 Forbidden ← no outbox row, no SMS

6. The Outbox Pattern

The outbox is the heart of the design. Rather than calling the vendor and publishing to Kafka on the request thread, the sender writes a durable row to a local relational table and returns. A background worker then drives that row through a state machine.

6.1 State machine



6.2 Why two steps

Splitting "call the vendor" (**PENDING**→**SENT**) from "publish the event" (**SENT**→**PUBLISHED**) is what guarantees no duplicate SMS. Once a row is **SENT**, a Kafka failure only ever retries the *publish* step — the vendor is never contacted again.

6.3 Backoff & dead-letter

Each failure doubles the retry delay (2s → 4s → 8s ... capped at 5 min) by stamping a future `nextAttemptAt`; the worker only picks up rows whose backoff has elapsed. After `max-attempts` (10) the row becomes **FAILED** and the worker stops touching it. The long backoff window (~13 min of continuous failure) means a transient Kafka outage is ridden out rather than wrongly quarantined.

Key property: the request thread does only a fast local DB write. A Kafka or vendor outage causes rows to queue in H2 — the service stays responsive and drains automatically on recovery.

7. Reliability & Idempotency

The system was stress-tested against concrete failure modes. Each exposed a real problem and a fix.

Failure mode	Resolution	Status
Event lost when Kafka is down (fire-and-forget on request thread)	Outbox pattern — durable H2 queue + async worker; service never blocks on Kafka	FIXED
Duplicate SMS on worker crash/retry	Two-step state machine + idempotency key passed to the vendor	FIXED
Duplicate events in MongoDB (at-least-once delivery)	Consumer upsert keyed by <code>eventId</code> + sparse unique index	FIXED
Poison messages retry forever	Exponential backoff + dead-letter (FAILED) + admin replay	FIXED
Open admin endpoints	API-key filter + gateway injection	FIXED

7.1 The idempotency chain

Vendor side: idempotency key = outbox row id
 → vendor returns cached result on replay → recipient messaged ONCE

Storage side: eventId = outbox row id (carried in the Kafka event)
 → consumer upserts on eventId → MongoDB converges to ONE document

Net guarantee: exactly-once to the recipient AND exactly-once in storage,
 even though Kafka in the middle is at-least-once by design.

The crash window: the vendor call and the SENT write are not atomic. If the worker crashes between them, the row is still PENDING and the vendor would be called again on restart — which is exactly why the idempotency key exists: the vendor dedups on it, so even that window cannot double-send.

8. Data Model

8.1 Outbox row (H2 — `outbox_events`)

Column	Type	Purpose
id	VARCHAR (PK)	UUID; also the idempotency key & event id
phoneNumber	VARCHAR	Recipient
message	VARCHAR(1000)	Body
status	ENUM	PENDING / SENT / PUBLISHED / FAILED
vendorStatus	VARCHAR	SUCCESS / FAILED (set at SENT)
vendorMessageId	VARCHAR	Vendor receipt id
errorReason	VARCHAR	Vendor business error
attempts	INT	Failures at the current step
nextAttemptAt	TIMESTAMP	Backoff gate — earliest eligible time
lastError	VARCHAR(500)	Last infrastructure failure (for inspection)
createdAt / updatedAt	TIMESTAMP	Lifecycle timestamps

8.2 Event on the wire / MongoDB document (`sms_events`)

```
{
  "eventId":      "c42490c2-ecd1-4bb6-ada2-2b9bfd378fe8",  // dedup key
  "phoneNumber":  "888888888888",
  "message":      "Hello there",
  "status":       "SUCCESS",
  "vendorMessageId": "VND-7f3a...",
  "errorReason":  null,
  "timestamp":    "2026-06-11T14:22:37+05:30"
}
```

Unique sparse index on `eventId` enforces one document per request.

8.3 Redis

A single Set, `blocked_users` , holding blocked phone numbers. Checked with `SISMEMBER` ($O(1)$) on every send.

9. Security

Admin endpoints (`/v1/sms/block/**` and `/v1/sms/outbox/**`) require an `X-API-Key` header validated by a Spring `OncePerRequestFilter` . Missing or invalid → `401` . Public endpoints (`/send` , `/health`) are open.

Browser (no secret) → Proxy/Gateway (holds key, injects X-API-Key) → Java filter → 
Attacker → Java filter →  `401`

The secret lives only in the gateway, never in browser code. This is correct for an internal operator; the production path is per-seller keys or JWT/OAuth2 with roles (see §14).

10. Technology Choices & Rationale

Technology	Used for	Why this tool
Redis	Blocklist (hot path)	In-memory, sub-ms; native Set type fits "is this number blocked?"
Kafka	Async event bus	Durable, replayable, decouples services; consumer groups for scale
MongoDB	Event history	Flexible JSON documents; write-heavy append/lookup workload
H2 (embedded)	Outbox table	Relational + transactional state machine; zero extra infra; Postgres-swappable
Zookeeper	Kafka coordination	Required by this Kafka version; not used directly by our code
Spring Boot (Java 21)	Sender	First-class Kafka/Redis/validation; LTS runtime
Go	Store	Lightweight concurrency (goroutines) for the consumer + HTTP server

11. API Reference

11.1 Java Sender (:8080)

Method	Path	Auth	Description
POST	/v1/sms/send	public	Accept an SMS request → 202 Accepted
GET	/v1/sms/health	public	Liveness
GET	/v1/sms/block/{phone}	key	Check if blocked
POST	/v1/sms/block/{phone}	key	Block a number
DELETE	/v1/sms/block/{phone}	key	Unblock a number
GET	/v1/sms/outbox	key	List recent outbox rows (monitor)
GET	/v1/sms/outbox/failed	key	List dead-lettered rows
POST	/v1/sms/outbox/{id}/replay	key	Requeue a FAILED row

11.2 Go Store (:8081)

Method	Path	Description
GET	/health	Liveness
GET	/v1/user/{phone}/messages	All stored events for a number

12. Configuration Reference

Setting	Default	Meaning
outbox.poll-interval-ms	1000	Worker scan cadence
outbox.max-attempts	10	Failures before dead-letter
outbox.backoff-base-ms	2000	First retry delay (doubles each attempt)
outbox.backoff-max-ms	300000	Retry-delay cap (5 min)
admin.api-key	env ADMIN_API_KEY	Shared admin secret
kafka.producer.retries	3	Producer auto-retry on transient failure
kafka.producer.max.block.ms	3000	Max time send() blocks when buffer is full
KAFKA_TOPIC	sms-events-topic	Topic name (both services)
GroupID (Go)	sms-store-group	Consumer group for offset tracking

13. Deployment

13.1 Demo topology

Infrastructure (Kafka, Zookeeper, MongoDB, Redis) runs via `docker-compose`. The Java and Go services and the proxy run directly so logs and restarts are visible during the demo.

13.2 Production topology

- All components containerized (services include their own runtime via multi-stage Dockerfiles).
- Services address dependencies by container/DNS name, not `localhost`.
- Orchestrated by Kubernetes for auto-restart, horizontal scaling, and zero-downtime rolling deploys.
- Kafka: 3 brokers, 3 replicas, multiple partitions, `acks=all` + idempotent producer.
- H2 → managed Postgres; secrets via a secret manager, not config files.

14. Known Limitations & Roadmap

Limitation	Impact	Planned resolution
Mock idempotency map is in-memory	Dedup state lost on JVM restart	Real vendor enforces idempotency server-side
No client-level idempotency	Two identical /send calls = two SMS	Client-supplied idempotency key on the request
Single partition & replica	No consumer scaling; broker crash loses data	3 replicas, N partitions, min.insync.replicas=2
No producer idempotence / acks=all	Lost ack can duplicate at the source	enable.idempotence=true + acks=all
Unbounded history query	Large result sets load fully into memory	Pagination (page/limit) + index on phoneNumber
Shared admin key	No per-user identity; can't ship to many sellers	Per-seller keys + tenant isolation (Twilio/Stripe model)
No outbox retention	PUBLISHED rows accumulate forever	Scheduled archival/cleanup job

All open items are configuration or additive changes — none require re-architecting the core. The outbox foundation, the idempotency chain, and the public/admin split are production-shaped.

15. Appendix & Glossary

Term	Meaning
Outbox pattern	Persist intent to a local table first, then process asynchronously, so acceptance is decoupled from work and nothing is lost.
Idempotency key	A stable per-request id ensuring a repeated operation has the same effect as doing it once.
At-least-once delivery	Kafka's guarantee that a message is delivered one or more times — consumers must be idempotent.
Dead-letter	A terminal FAILED state for messages that exhaust retries, set aside for inspection/replay.
Consumer group	A named set of consumers sharing a topic's partitions and tracking processed offsets together.
Backoff	Increasing the delay between retries to ride out transient outages.

Summary. A two-service, event-driven SMS platform whose transactional outbox makes acceptance durable and decoupled, whose end-to-end idempotency chain guarantees exactly-once delivery to both the recipient and the store, and whose dead-letter queue and API-key gate make it operable and protected. The remaining work is scaling and multi-tenancy — the foundation is sound.